

Mathematics for AI

Imre Fekete

feketei@ceri.edu

Motivation

Big picture: AI = mimic human behaviour

↳ ML = learn about the world

↳ DL = extract patterns using NNs (neural networks) /see the pdf/

The rise of AI:

↳ Everyday impact ~ LLMs such as ChatGPT (OpenAI) and Gemini (Google DeepMind)

↳ Scientific impact ~ publications, major scientific prizes

↳ Industrial impact ~ investments, new companies

/for further details and examples on each point, see the pdf/

In this course we will explore a tiny part of DL. But what exactly is DL?

ML is a subset of AI that focuses on algorithms that improve their performance on a task as more data becomes available. These methods typically start from a general model that is trained on data to perform a specific task. When this model is a (deep) NN, the approach is called DL. NNs are simply flexible functions fit to data.

Seminal contributions

- 1943 - McCulloch and Pitts: computational models inspired by biological neurons
"A logical calculus of the ideas immanent in nervous activity", The Bulletin of Mathematical Biophysics 5.4, pp. 115-133
- 1966 - Ivakhneko and Lapa: data-driven training of NNs
"Cybernetic predicting devices" Tech. Report, Purdue University
- 1987 - Fukushima: originated the ancestor of our current convolutional NNs
"NN model for selective attention in visual pattern recognition and associative recall"
Applied Optics 26:23, pp. 4385-4392
- 1989 - LeCun et al: introducing backpropagation as a training mechanism for NNs
"Backpropagation applied to handwritten zip code recognition"
Neural Computation 1:4, pp. 541-551

However, these efforts didn't lead to practical breakthroughs, and NNs were largely seen as academic curiosity at the time. So why is DL so popular today?

↳ rapidly increasing amounts of data

/see the pdf, left panel/

↳ hardware improvements ~ GPUs

/see the pdf, middle panel/

↳ software improvements ~ mature DL frameworks

/see the pdf, ~~left~~ right panel/

~ better training algorithms

/see all of them/

~ deeper and more sophisticated algorithms

Despite over a decade of remarkable experimental success, a solid theoretical understanding of DL remains limited.

I The Basics

DL is commonly organized around three main learning paradigms:

- Supervised learning

- ↳ Regression
- ↳ Classification

"Explicit labels"

/visualization, see the pdf/

- Unsupervised learning

- ↳ Feature / representation learning
- ↳ Autoregressive modeling
- ↳ Generative models

"No labels, but a learning objective"

- Reinforcement learning

- ↳ Policy learning
- ↳ Value function learning
- ↳ Model-based / model-free RL

"No labels, only reward and feedback"

Due to time constraints, we will focus only on certain cases of supervised learning.

Section 1: The supervised learning problem

This is a ML paradigm where each input has a known output but the mapping between them is unknown. The goal is to infer this mapping from data so it generalizes to new inputs.

Given an unknown function $f: X \rightarrow Y$ between spaces X and Y , find a good approximation of f using only a dataset of N samples:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N \text{ with } y_i = f(x_i) \text{ for all } i = 1, \dots, N$$

Notions:

↳ $X \sim$ feature space

↳ $\{x_i\} \sim$ feature vectors

↳ $Y \sim$ label space

↳ $\{y_i\} \sim$ labels

Example: /Classification/

Problem: image classification

Input: images of cats and dogs

Output: class probabilities

$$\leadsto X = [0, 1]^{3 \times H \times W}$$

(RGB image)

$$\leadsto Y = [0, 1]^2$$

(class probabilities)

$\leadsto f$ the classifier maps to Y

Practical remarks

↳ Image input

- Raw image data: $RGB \in \{0, \dots, 255\}^3$ integer values

- DL input: $RGB \in [0, 1]^3$ normalized fl values, $x_{\text{normalized}} = \frac{x}{255}$ "color intensity"

↳ Image notation convention

- DL / PyTorch (Meta) $\sim (C, H, W)$ C stands for channels

- TensorFlow / Keras (both Google) $\sim (H, W, C)$

- Math regarding $H \times W \sim$ row $\times$ column

/Example: see the pdf/

What do the notations $[0, 1]^{3 \times H \times W}$ and $[0, 1]^2$ mean exactly?

Important remarks

- It is not related to the Cartesian product

$$A \times B = \{(a, b) \mid a \in A \text{ and } b \in B\}$$

ordered pair element of the set

So $[0, 1]^2 = [0, 1] \times [0, 1]$ is the unit square in this Cartesian setting

$$\text{Furthermore, } [0, 1]^{3 \times H \times W} = \prod_{i=1}^3 \prod_{j=1}^H \prod_{k=1}^W [0, 1]$$

Main problem: we don't have algebraic operations like addition and scalar multiplication

- It is related to Tensor products

Def: If V and W are vector spaces, then the tensor product $V \otimes W$ is a vector space s.t

1) the basis $\{v_i \otimes w_j \mid v_i \text{ basis of } V, w_j \text{ basis of } W\}$

2) $\forall \alpha, \beta \in \mathbb{R}, v_1, v_2 \in V, w_1, w_2 \in W$:

$$\text{Bilinearity} \begin{cases} (\alpha v_1 + \beta v_2) \otimes w = \alpha (v_1 \otimes w) + \beta (v_2 \otimes w) & \leftarrow \text{Lin. in the 1st factor if the 2nd is fixed} \\ v \otimes (\alpha w_1 + \beta w_2) = \alpha (v \otimes w_1) + \beta (v \otimes w_2) & \leftarrow \text{Lin. in the 2nd factor if the 1st is fixed} \end{cases}$$

$$\text{So } \otimes : \underbrace{V \times W}_{\text{Cartesian product}} \rightarrow V \otimes W$$

Bilinearity ensures that lin. combination of basis vectors consistently produce a new vector in the tensor product space.

Let's have an illustrative example for a 2×4 pixel image

$$\left. \begin{array}{l} \text{Channels } C = \mathbb{R}^3 (R, G, B) \\ \text{Rows / Height: } H = \mathbb{R}^2 \\ \text{Columns / Width: } W = \mathbb{R}^4 \end{array} \right\} \text{Tensor product vector space } C \otimes H \otimes W \cong \mathbb{R}^{3 \cdot 2 \cdot 4} = \mathbb{R}^{24}$$

"isomorphic to"

Bases:

$$e_c = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad e_i = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad e_j = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$X = \sum_{c=1}^3 \sum_{i=1}^2 \sum_{j=1}^4 X_{cij} e_c \otimes e_i \otimes e_j \in C \otimes H \otimes W \xleftrightarrow{\text{one-to-one lin. correspondence}} x \in \mathbb{R}^{24}$$

vector space tensor product element vector space

Every vector in $\mathbb{R}^{24}$ can be uniquely written as a lin. comb. of the $C \otimes H \otimes W$ basis vectors

$$X \text{ image} \sim \begin{bmatrix} \boxed{3} & 5 & 4 & 2 \\ 7 & 1 & 6 & 0 \end{bmatrix} \quad \begin{bmatrix} \boxed{1} & 2 & 0 & 6 \\ 3 & 4 & 0 & 2 \end{bmatrix} \quad \begin{bmatrix} \boxed{0} & 1 & 3 & 1 \\ 0 & 2 & 5 & 3 \end{bmatrix}$$

R G B Pixel (1,1) RGB value

$$\sim \begin{bmatrix} [3, 1, 0] & [5, 2, 1] & [4, 0, 3] & [2, 6, 1] \\ [7, 3, 0] & [1, 4, 2] & [6, 0, 5] & [0, 2, 3] \end{bmatrix}$$

array repr. in our programming mind

$$Y \text{ image} \sim \begin{bmatrix} [2, 3, 1] & [0, 5, 2] & [1, 4, 0] & [3, 0, 5] \\ [0, 1, 6] & [2, 2, 1] & [1, 3, 2] & [5, 1, 0] \end{bmatrix} \sim \begin{bmatrix} 2 & 0 & 1 & 3 \\ 0 & 2 & 1 & 5 \end{bmatrix} \quad \begin{bmatrix} 3 & 5 & 4 & 0 \\ 1 & 2 & 3 & 1 \end{bmatrix} \quad \begin{bmatrix} 1 & 2 & 0 & 5 \\ 6 & 1 & 2 & 0 \end{bmatrix}$$

R G B

$X + Y$?

First let's represent $X + Y = Z$ as $Z = \sum_{c,i,j} (X_{cij} + Y_{cij}) (e_c \otimes e_i \otimes e_j)$

So for the pixel (1,1) we have

$$Z_{11} = \sum_{c=1}^3 (X_{c11} + Y_{c11}) (e_c \otimes e_1 \otimes e_1) = (X_{111} + Y_{111}) (e_1 \otimes e_1 \otimes e_1) + \dots + (X_{311} + Y_{311}) (e_3 \otimes e_1 \otimes e_1)$$

- 2 -

$$= \underbrace{5(e_1 \otimes e_1 \otimes e_1)}_{3+2} + \underbrace{4(e_2 \otimes e_1 \otimes e_1)}_{[1+3]} + \underbrace{1(e_3 \otimes e_1 \otimes e_1)}_{0+1}$$

Now apply the formula for flattening which guarantees the one-to-one lin. correspondence (isomorphism)

Let $T \in \mathbb{R}^{d_1 \times \dots \times d_N}$ be an N dimensional tensor: $T = \sum_{i_1=1}^{d_1} \dots \sum_{i_N=1}^{d_N} T_{i_1, \dots, i_N} e_{i_1} \otimes \dots \otimes e_{i_N}$, then

$$\text{flattening-index} = 1 + \sum_{k=1}^N (i_k - 1) \prod_{l=k+1}^N d_l$$

So every tensor product basis element $e_{i_1} \otimes \dots \otimes e_{i_N}$ is mapped uniquely to a 1D vector component.

In our case is $N=3$

$$d_1 = C = 3$$

$$d_2 = H = 2$$

$$d_3 = W = 8$$

$$\text{index} = 1 + (C-1) \cdot \underbrace{8}_{H \cdot W} + (i-1) \cdot \underbrace{8}_{W} + (j-1)$$

Thus, $z_{11} = [5, 0, \dots, 0, \underbrace{4}_{H}, 0, \dots, 0, \underbrace{1}_{W}, 0, \dots, 0]^T$. Similarly, by applying the previous indexation to all pixels

we derive the following vector $z_{\text{flat}} = [5, 5, 5, 7, 3, \dots, 66, 3, 7, 3]^T$. /see the pdf for the other values/

So $z \in C \otimes H \otimes W \cong z_{\text{flat}} \in \mathbb{R}^{C \cdot H \cdot W}$

Technical remarks:

In our example $3 \times 2 \times 4$ tensor $\leadsto$ 24 dim. flat vector

$\rightarrow$ Tensor form: • From the user perspective it is easier to visualize
• Visual output in PyTorch, TensorFlow is an array

$\rightarrow$ Flat vector: • PyTorch and TensorFlow internally store the tensor as a 24 dim 1D array

- This form is nice since physical memory consists of consecutive cells. Linear memory is important due to efficient parallel access, simpler indexing and reduced cache missing reasons. CPUs have fewer cores and less bandwidth for massive parallelism so the GPU gains most from this structure
- All lin. alg. operations (including activations, convolutions what we discuss later) are computed on this vector (lin. array).

For the end of the example

$\rightarrow X = \mathbb{R}^3 \otimes \mathbb{R}^H \otimes \mathbb{R}^W$ / handy notation $\mathbb{R}^{3 \times H \times W}$ / $\sim$ tensor product space

$x \in X \sim$ element, i.e. the actual picture $\sim$ tensor

In practice $x \in X \cong x_{\text{flat}} \in \mathbb{R}^d$, where $d = 3 \cdot H \cdot W$

$\rightarrow Y = \mathbb{R}^2$

$\hat{Y} = \{p \in \mathbb{R}^2 \mid p_1 \geq 0, p_2 \geq 0, p_1 + p_2 = 1\}$. $\hat{Y} \subset \mathbb{R}^2 = Y$ and requires an additional consideration $[0, 1]^2$

$\rightarrow f: X \rightarrow Y$ is the classifier which maps to $\mathbb{R}^2$ we need an extra idea to convert it to $\hat{Y}$. All of these advanced f should result in a vector space structure to do optimization. We will see them later.

Remark: problem with $\hat{Y}$. It is not a vector space ($p, q \in \hat{Y}$, then in general $p+q \notin \hat{Y}$). [In this example it is just a simplex.] It would cause a problem during optimization, since numerical algorithms require algebraic operations and an addition would leave the simplex $\hat{Y}$ which would be a problem.

The general approach consists 3 main steps:

- 1) Choose a model $F: X \times W \rightarrow Y$ parametrized by a parameter space W (W for weights since that is how parameters in NNs are often referred to).
- 2) We need to quantify the quality of the model's output, so we need to choose a loss function $\ell: Y \times Y \rightarrow \mathbb{R}$ where $\ell(y_1, y_2)$ indicates "how different" y_1 and y_2 are.
- 3) Based on the dataset and the loss function choose $w \in W$ s.t. $F(\cdot, w)$ is the "best" possible approximation of the target function f .

This general approach still leaves open questions.

- 1) How to choose the model?
- 2) How to choose the loss function?
- 3) How to optimize?

These questions have no canonical answers and are typically addressed through heuristics and trial-and-error.

- 1) We will naturally choose NVs as our primary model which we will discuss in more detail later.
- 2) A loss function is a function $\ell: \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ it works much like a metric but it is less restrictive
- 3) Based on our previous studies a gradient descent based algorithm would be obvious

Let's focus on a little bit on 2).

↳ usually but not always $\mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}^+$

↳ the minimum should exist s.t. $\min \ell(y_0, y)$ is well posed

↳ diff. (a.e. at least) would be ^{$y \in \mathcal{Y}$} necessary since we want to do gradient descent

↳ metric properties like identity of indiscernibles, symmetry and triangle ineq. need not hold

Let's focus on a little bit on 3)

↳ finding the best $w \in W$ typically by minimizing the loss on the dataset, i.e. find

$$w^* = \arg \min_{w \in W} \sum_{i=1}^N \ell(f(x_i; w), y_i)$$

Example / Linear least squares - Regression /

$f(x; w) := \sum_{j=1}^n w_j f_j(x)$ and $\ell(y_1, y_2) = |y_1 - y_2|^2$ for some basis functions $\{f_j\}_{j=1}^n$

Given: $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N \sim$ we don't exclude the possibility that some x_i coincide i.e. there are multiple measurements at the same point.

$f_j(x) = x^{j-1} \sim$ polynomial regression

This problem can be represented as a lin. system. Namely,

$$A = (a_{ij}) = (f_j(x_i)) \in \mathbb{R}^{N \times n}, \text{ i.e. } A = \begin{pmatrix} 1 & x_1 & x_1^2 & \dots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \dots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \dots & x_N^{n-1} \end{pmatrix} \in \mathbb{R}^{N \times n}$$

$w := (w_1, w_2, \dots, w_n)^T$ and $f = (y_1, y_2, \dots, y_N)^T$, i.e. $w \in \mathbb{R}^n$, $f \in \mathbb{R}^N$. So we have $Aw = f$.

$$\text{Technically speaking } w^* = \min_w \sum_{i=1}^N \sum_{j=1}^n (w_j f_j(x_i) - y_i)^2 = \min_w \|Aw - f\|_2^2$$

By applying the basic multivariable calculus technique for finding local extrema the sought minimum can occur where $A^T A w = A^T f$ // (Gauss) normal equation /

◇

/ For concrete examples see the Jupyter notebook /

Conclusions based on the regression examples

- Underfitting: The model is too simple for the data. It cannot capture the true pattern so it performs poorly on both training and new data.
- Overfitting: The model is too complex and learns the noise in the data as well. It performs very well on the training data but poorly on new unseen data.

/ see the pdf /

The "best" $w \in W$ isn't necessarily the one that minimizes dataset loss as overfitting is common. This motivates regularization. Parameter regularization, a technique from regression often used in NNs adds a penalty to the loss to discourage undesirable parameter values. The modified opt. problem is

$$w^* = \operatorname{argmin} \sum_{i=1}^N \ell(f(x_i; w), y_i) + \lambda C(w)$$

with $\lambda > 0$ and $C: W \rightarrow \mathbb{R}^+$ to penalize complexity in some fashion.

Example / Regression - revisited /

The matrix ATA is symmetric and positive semidefinite.

- symmetric: $(ATA)^T = A^T (AT)^T = ATA$
- positive semidefinite: $\forall v \in \mathbb{R}^n: v^T (ATA) v = \langle ATA v, v \rangle = \langle Av, Av \rangle = \|Av\|_2^2 \geq 0$ since this holds for all v ATA is positive semidefinite

The matrix ATA has nonnegative eigenvalues

Proof: Let $\hat{\lambda} \in \mathbb{C}$ be an eigenvalue of ATA and let $v \in \mathbb{C}^n \setminus \{0\}$ be a corresponding eigenvector
 $ATA v = \hat{\lambda} v$

$$\langle ATA v, v \rangle = \langle v, ATA v \rangle = \langle v, \hat{\lambda} v \rangle = \hat{\lambda} \langle v, v \rangle \Rightarrow \langle \hat{\lambda} v, v \rangle = \hat{\lambda} \langle v, v \rangle = \bar{\hat{\lambda}} \langle v, v \rangle$$

symm. scales product property

Since $\langle v, v \rangle > 0 \Rightarrow \hat{\lambda} = \bar{\hat{\lambda}}$ so the eigenvalues are real. ($\lambda \in \mathbb{R}$)

Furthermore, $v^T ATA v = v^T (\hat{\lambda} v) = \hat{\lambda} v^T v = \hat{\lambda} \|v\|_2^2$. Since ATA is pos. semidefinite $v^T ATA v \geq 0$
 since $v \neq 0$ and $\|v\|_2^2 > 0 \Rightarrow \hat{\lambda} \geq 0$. □

Remark: ATA is pos. def. ($\lambda > 0$) iff A has full column rank ($\operatorname{rank}(A) = n$)

In practice even if $N > n$ high degree poly. features may make ATA nearly singular (small eigenvalues) causing numerical instability.

Underfitting: $n \ll N$ or low degree polynomial $\leadsto$ cannot capture the true pattern $\leadsto$ high bias

Overfitting: $n \approx N$ or very high degree poly $\leadsto$ learns noise $\leadsto$ high variance

Bias-variance trade-off: increasing model complexity reduces bias but increases variance.

Regularization helps balance this trade-off. To stabilize the solution and reduce overfitting we add a regularization term: $\|Aw - f\|_2^2 + \lambda \|w\|_2^2$, $\lambda > 0$.

Tikhonov / L_2 regularization term.

The parameter λ controls the trade-off between data fitting (small error) and model simplicity (small coefficients). The regularized solution satisfies the modified normal equation

$$(ATA + \lambda I) w = A^T f$$

This guarantees that the matrix $ATA + \lambda I$ is pos. def. so all eigenvalues are pos (the matrix is also symm) even if ATA is singular or nearly singular. Numerically small eigenvalues of ATA are "lifted" by λ .

Stabilizing inversion.

Role of λ

$\hookrightarrow$ small $\lambda \approx 0$: solution close to ordinary least squares $\leadsto$ may overfit, unstable if ATA is ill-conditioned

$\hookrightarrow$ large λ : coefficients are strongly damped $\leadsto$ smoother model, higher bias, less variance, possible underfitting

Choosing λ in practice

$\hookrightarrow$ the true noise level is unknown so λ cannot be derived analytically

$\hookrightarrow$ it is selected empirically via cross-validation

The corresponding steps are:

- 1) Split the data into training and validation sets
- 2) Train the model for several candidate λ values on the training set
- 3) Evaluate prediction error on validation data
- 4) Choose λ that minimizes validation error

This ensures the model achieves the best generalization performance.

Practical considerations on N and n

- ↳ Stable regression generally requires $N > n$ (rule of thumb $N > 5 \cdot M$ exact const. depends on noise and feature correlations)
- ↳ If $N \times n$, $A^T A$ may be ill-conditioned $\leadsto$ regularization recommended
- ↳ If $N < n$, $A^T A$ is singular $\leadsto$ reg. is necessary.

◇

/For concrete examples see the jupyter notebook/

Section 2: Basic network types, activation functions and universality

Artificial NNs are inspired by biology and consist of interconnected artificial neurons that exchange signals.

/see the pdf/

ANNs are made of artificial neurons that usually map real valued inputs to outputs. The most common model neuron is given by an affine transformation followed by a nonlin. function. Namely, it has the form

$$y = \sigma(Ax + b), \text{ where}$$

- $x \in \mathbb{R}^n$ is the input signal; $y \in \mathbb{R}^m$ is the output signal
- $A \in \mathbb{R}^{m \times n} \sim$ linear weights (weight matrix)
- $b \in \mathbb{R}^m \sim$ bias (bias vector)
- $\sigma \sim$ activation function (transfer function or nonlinearity)
Typically pointwise (elementwise) application

Remark: Inputs and outputs need not span the real numbers. Depending on the application we may have:

↳ $\{0, 1\}$ (binary); $\mathbb{R}^+$; $[0, 1]$ (prob. for example); $\mathbb{C}$

We start with shallow networks because their simplicity helps build intuition and can be seen as single layers of deep networks. Let us consider a shallow network with scalar input and output. Let $w = (a, b, c) \in W = (\mathbb{R}^3)$ be our set of parameters for some $N \in \mathbb{N}$. Then we define our model

$$F: \mathbb{R} \times W \rightarrow \mathbb{R} \text{ as } F(x, w) := \sum_{i=1}^N c_i \sigma(a_i x + b_i).$$

/see the pdf/

A quintessential example of a deep NN is the feedforward network. In this architecture, neurons are connected without forming cycles, and the network can be thought of as a stack of shallow networks where the output of each layer naturally feeds into the next.

/see the pdf/

Let us formalize this construction. Let $L \in \mathbb{N}$ and $N_0, N_1, \dots, N_{L+1} \in \mathbb{N}$. Let $\sigma_1, \dots, \sigma_L$ be activation functions (we will assume to be scalar functions that we apply pointwise for the moment), let $F_i: \mathbb{R}^{N_{i-1}} \rightarrow \mathbb{R}^{N_i}$ be affine transforms given by $F_i(x) := A_i x + b_i$, where $A_i \in \mathbb{R}^{N_i \times N_{i-1}}$ and $b_i \in \mathbb{R}^{N_i}$, $i = 1, \dots, L+1$.

Then we call

$$W: \mathbb{R}^{N_0} \rightarrow \mathbb{R}^{N_{L+1}} \text{ given by } W := F_{L+1} \circ \sigma_L \circ F_L \circ \dots \circ \sigma_1 \circ F_1$$

a feedforward NN.

↳ $L \sim$ hidden layers

↳ $N_i \sim$ width of layer i ; $\max_{1 \leq i \leq L} \{N_i\} \sim$ also called the width of the network

We usually inputs with x , output with y and if necessary the intermediate results with $z^{(i)} \in \mathbb{R}^{N_i}$ as follows:

$$z^{(0)} = x, \quad z^{(i)} = \sigma_i(F_i(z^{(i-1)})), \quad i = 1, \dots, L+1 \quad / \text{see the pdf} /$$

In the following, we consider the previously defined shallow network, and after examining two specific activation functions, we arrive at the proof of a universality type theorem.

One of the earliest and historically significant choices of activation function is the Heaviside function which is

$$\#(x) := \mathbb{1}_{x \geq 0}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{else} \end{cases}$$

A type $y = \sigma(Ax + b)$ neuron with Heaviside activation is called a perceptron. We don't like it practice since it cannot be used by the learning procedure (typically it is gradient based and $\#(x)$ is not differentiable at $x=0$ and has 0 derivative elsewhere).

Let's see what we can do with it. Let $w \in \mathbb{R}^n$, $b \in \mathbb{R}$, then a neuron with input $x \in \mathbb{R}^n$ and output $y \in \mathbb{R}$ would look like

$$y(x) = \#(w^T x + b) = \begin{cases} 1 & \text{if } w^T x \geq -b \\ 0 & \text{if } w^T x < -b \end{cases}$$

This is a linear binary classifier on $\mathbb{R}^n$ where $w^T x = -b$ divide the space into 0 and 1 regions.

Example: / Boolean gate /

Let $x_1, x_2 \in \{0, 1\}$, and let $\text{AND}(x_1, x_2) = \#(x_1 + x_2 - 1.5)$. Then we have

x_1	x_2	$\text{AND}(x_1, x_2)$
0	0	0
0	1	0
1	0	0
1	1	1

The Heaviside function is a scalar (pointwise) activation. For vectors or matrices it's applied elementwise, i.e.

$\#(x) = [\#(x_1) \ \#(x_2) \ \dots]^T$. Rectified Linear Unit (ReLU), probably the most widely used in modern NNs, is defined as

$$\sigma(x) = \text{ReLU}(x) = \max\{0, x\} \quad \text{where } x \in \mathbb{R}. \quad / \text{see the pdf} /$$

ReLU can be seen as a cont. generalized version of the Heaviside function. It behaves like a step at 0 but outputs a cont. unbounded value for pos. inputs making it suitable for gradient based learning.

In the following we will examine the theoretical reasons behind ReLU's popularity.

Let us consider a shallow NN where $x \in [0, 1]$. This is not much of a restriction in practice since real world data is typically bounded. Let us examine the types of functions the network can represent. Its output is a lin. comb. of functions of the form $x \mapsto \sigma(ax + b)$ which take different shapes depending on a . /see the pdf/

All 3 func. types are piecewise lin. so any lin. comb. is also piecewise lin. Thus, the model $\#$ is essentially a parameterization of piecewise lin. functions on $[0, 1]$. Can a shallow ReLU network represent any such function?

Let $f: [0, 1] \rightarrow \mathbb{R}$ be piecewise lin. function with N pieces. We will denote the inflection points with $0 = \beta_0 < \beta_1 < \dots < \beta_{N+1} = 1$ and the slopes (i.e. the constant derivative of each piece) with $\alpha_1, \dots, \alpha_N$ /see the pdf/

Now we choose

$$f(x) = \sum_{i=1}^{N+1} c_i \sigma(a_i x + b_i) \quad \text{where } \sigma = \text{ReLU} \text{ and}$$

$$\begin{aligned} a_{N+1} &= 0 & b_{N+1} &= 1 & c_1 &= \alpha_1 \\ a_1, \dots, a_N &= 1 & b_i &= -\beta_i, i=1, \dots, N & c_i &= \alpha_i - \alpha_{i-1} \quad i=2, \dots, N \\ & & & & c_{N+1} &= f(0) \end{aligned}$$

This turns our model

$$F(x) = f(0) + \alpha_1 x + \sum_{i=2}^N (\alpha_i - \alpha_{i-1}) \sigma(x - \beta_i).$$

Why? Since

↳ $C_{N+1} \rightarrow \sigma(\alpha_{N+1}x + b_{N+1}) = \sigma(0 \cdot x + 1) = \text{ReLU}(1) = 1$, so this neuron contributes a constant $C_{N+1} = f(0)$ to the output. This sets the starting value of the function: $F(0) = f(0)$

↳ The first lin. piece is $\sigma(x - \beta_1) = \sigma(x - 0) = \sigma(x) = \text{ReLU}(x) = x$ for $x \in [\beta_1, \beta_2]$

All other term vanishes: $\sigma(x - \beta_i) = 0$ for $i \geq 2$.

The output is $F(x) = f(0) + \alpha_1 x = f(0) + \alpha_1 x$ which matches $f(x)$ exactly on $[\beta_1, \beta_2]$.

↳ The next piece: for $x \in [\beta_2, \beta_3]$: the first term ReLU term beyond the first activates: $\sigma(x - \beta_2) > 0$.

$$\text{The output is } F(x) = f(0) + \alpha_1 x + (\alpha_2 - \alpha_1) \sigma(x - \beta_2) = \frac{f(0) + \alpha_1 \beta_2}{x - \beta_2} + \alpha_2 (x - \beta_2), \quad x \in [\beta_2, \beta_3]$$

↳ continuing along the x-axis:

every time we cross an inflection point β_i the next ReLU term activates: $\sigma(x - \beta_i) > 0$.

The slope changes by $c_i = \alpha_i - \alpha_{i-1}$ bending the line to match $f(x)$.

So, we have

$$F(x) = \begin{cases} f(0) + \alpha_1 x, & x \in [\beta_1, \beta_2] \\ f(\beta_2) + \alpha_2 (x - \beta_2), & x \in [\beta_2, \beta_3] \\ \vdots \\ f(\beta_{N-1}) + \alpha_N (x - \beta_{N-1}), & x \in [\beta_{N-1}, \beta_N] \end{cases}$$

Remark: On the last interval $[\beta_N, \beta_{N+1}]$, no new ReLU activates. The slope α_N from the previous segment continues, so $F(x)$ exactly matches $f(x)$ up to $\beta_{N+1} = 1$.

Thus, $F(x) = f(x)$ for $x \in [0, 1]$. Hence, we may conclude that shallow scalar ReLU networks exactly represent piecewise lin. functions on compact intervals which can approximate any cont. func. on $[0, 1]$ as stated in the next theorem.

Theorem: Let $f \in C([0, 1], \mathbb{R})$ then for all $\epsilon > 0$ there exists a piecewise lin. func. F so that $\sup_{x \in [0, 1]} |f(x) - F(x)| < \epsilon$.

To prove the theorem we will need technical tools so let us take a short intermezzo.

Continuity and uniform continuity

Def: Let $f: D \subset \mathbb{R} \rightarrow \mathbb{R}$ and $x_0 \in D$. The function f is cont. at x_0 if

$$\forall \epsilon > 0 \exists \delta > 0 : |x - x_0| < \delta \Rightarrow |f(x) - f(x_0)| < \epsilon.$$

Remarks:

↳ f is cont. on D if it is cont. at every point $x_0 \in D$

↳ δ is allowed to depend on x_0

/see the pdf/

Def: A function $f: D \rightarrow \mathbb{R}$ is uniformly cont. on D if

$$\forall \epsilon > 0 \exists \delta > 0 : \forall x_1, x_2 \in D \quad |x_1 - x_2| < \delta \Rightarrow |f(x_1) - f(x_2)| < \epsilon.$$

Remarks:

↳ the same δ must work for all points simultaneously

↳ δ depends only on ϵ (not on the location in the domain)

/see the pdf/

Example: Consider the function $f(x) = \frac{1}{x}$, $x \in (0, 1]$

- f is cont. on $D \Rightarrow f$ is diff. on $D \Rightarrow f$ is cont. on D
- f is not uniformly cont. on D

Proof: $\nearrow$ for contradiction: assume that f is uniformly cont. on D . By definition for $\epsilon = 1 \exists \delta > 0 : \forall x_1, x_2 \in D$

$$|x_1 - x_2| < \delta \Rightarrow \left| \frac{1}{x_1} - \frac{1}{x_2} \right| < 1. \text{ Now } x_1 = \frac{\delta}{2} \text{ and } x_2 = \frac{\delta}{4}.$$

$$\text{Then } |x_1 - x_2| = \frac{\delta}{4} < \delta. \text{ However, } \left| \frac{1}{x_1} - \frac{1}{x_2} \right| = \left| \frac{2}{\delta} - \frac{4}{\delta} \right| = \frac{2}{\delta} > 1 \text{ for all sufficiently small } \delta.$$

□

Conclusions:

- Uniform cont. $\Rightarrow$ cont.
- Cont. $\nRightarrow$ uniform cont.

However, cont. + compact domain $\Rightarrow$ uniform cont. / Heine - Cantor theorem /

Def: A set K is compact if every open cover of K has a finite subcover.

Fortunately, this general topology notion has a very concrete characterization in $\mathbb{R}$ (and also in $\mathbb{R}^n$).

Theorem /Heine - Borel/: Let $K \subset \mathbb{R}$. Then K is compact if and only if K is closed and bounded.

Remark: $K = [0, 1]$ is compact since $[0, 1]$ is closed and bounded.

Theorem /Simplified Heine - Cantor/: Let $f \in C([0, 1])$. Then f is uniformly cont. on $[0, 1]$.

Proof: $\nearrow$ Assume that f is not uniformly cont. on $[0, 1]$. Then $\exists \epsilon_0 > 0$ s.t. $\forall \delta > 0 \exists x_1, x_2 \in [0, 1]$ with

$$|x_1 - x_2| < \delta \text{ that } |f(x_1) - f(x_2)| \geq \epsilon_0. \text{ Let } \delta_n = \frac{1}{n}. \text{ For each } n \exists x_n, y_n \in [0, 1] \text{ such that}$$

$$|x_n - y_n| < \frac{1}{n} \text{ and } |f(x_n) - f(y_n)| \geq \epsilon_0.$$

Since we have $[0, 1]$, then the sequence (x_n) has a convergent subsequence: $x_{n_k} \rightarrow x^* \in [0, 1]$

/ This is the Bolzano - Weierstrass theorem: every bounded seq. in $\mathbb{R}$ has a conv. subseq. /

Example: $x_n = (-1)^n, n = 1, 2, \dots$

Bounded: $-1 \leq x_n \leq 1 \quad \forall n$

Not conv.: $x_1 = -1, x_2 = 1, x_3 = -1, x_4 = 1, \dots$

But all even indices $x_{2n} = 1 \rightarrow 1$
or all odd indices $x_{2n-1} = -1 \rightarrow -1$

} conv. subseq.

Remark: Obv. if x_n is conv. then we have conv. subseq.

Because $|x_{n_k} - y_{n_k}| \rightarrow 0$ we have $y_{n_k} \rightarrow x^*$. By continuity of f at x^* , we have $f(x_{n_k}) \rightarrow f(x^*)$ and

$f(y_{n_k}) \rightarrow f(x^*)$. Hence, $|f(x_{n_k}) - f(y_{n_k})| \rightarrow 0$ (contradicts to $|f(x_{n_k}) - f(y_{n_k})| \geq \epsilon_0 > 0$)

□

So the " $f \in C([a, b]) \Rightarrow f$ is uniformly cont. on $[a, b]$ " is the key result used often implicitly in approximation theorems.

We are finally ready to prove the theorem stated before this intermezzo part.

Theorem Let $f \in C([0, 1], \mathbb{R})$ then for all $\epsilon > 0$ there exists a piecewise lin. func. F so that $\sup_{x \in [0, 1]} |f(x) - F(x)| < \epsilon$

Proof: Since $f \in C([0, 1])$, due to Heine - Cantor f is uniformly cont. on $[0, 1]$, i.e. for the given $\epsilon > 0 \exists \delta > 0$.

$$\forall x_1, x_2 \in [0, 1] : |x_1 - x_2| < \delta \Rightarrow |f(x_1) - f(x_2)| < \epsilon/2.$$

Choose $N \in \mathbb{N} \setminus \{0, 1\}$ ($N \geq 2$ integer) s.t. $\frac{1}{N} < \delta$ and define $x_i = \frac{i}{N}$, $i = 0, 1, \dots, N$. Then each subinterval has length $x_i - x_{i-1} = \frac{1}{N} < \delta$.

So the distance between any 2 consecutive nodes is less than δ which ensures

$$|f(x_i) - f(x_{i-1})| < \frac{\epsilon}{2}.$$

Define $F: [0,1] \rightarrow \mathbb{R}$ by $F(x) = f(x_{i-1}) + \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}} (x - x_{i-1})$, $x \in [x_{i-1}, x_i]$

/or $\sum \mathbb{1}_{[x_{i-1}, x_i]}(x) \left(\frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}} (x - x_{i-1}) + f(x_{i-1}) \right)$ / and set $F(\frac{x_N}{1}) = f(\frac{x_N}{1})$

Then we see that $F(x_i) = f(x_i) \forall i=0, \dots, N$ and F is linear on each subinterval.

Let $x \in [0,1]$ be arbitrary. Then $x \in [x_{i-1}, x_i]$ for some i .

$$|f(x) - F(x)| \leq \underbrace{|f(x) - f(x_i)|}_I + \underbrace{|f(x_i) - F(x)|}_II$$

In case of I: x and x_i are most one subinterval length apart, so $|x - x_i| < \frac{1}{N} < \delta$. Therefore, by uniform cont.

$$|f(x) - f(x_i)| < \frac{\epsilon}{2}$$

In case of II: on $[x_{i-1}, x_i]$, F is linear connecting $f(x_{i-1})$ and $f(x_i)$. So the max. difference between $F(x)$ and $f(x_i)$ occurs at x_{i-1} . Thus, $|F(x) - f(x_i)| \leq |f(x_{i-1}) - f(x_i)| < \epsilon/2$.

Therefore, $|f(x) - F(x)| \leq \underbrace{\frac{\epsilon}{2}}_I + \underbrace{\frac{\epsilon}{2}}_II = \epsilon$. This holds for all $x \in [0,1]$ so $\sup_{x \in [0,1]} |f(x) - F(x)| < \epsilon$. $\square$

What we have proved so far

- Shallow scalar ReLU networks represent all piecewise lin. funct. on $[0,1]$

- Piecewise lin. func. are dense in $C([0,1])$ w.r.t. the supremum norm.

↳ dense: when a class of functions is dense in another; any function in the larger class can be approximated arbitrarily closely by a function from the smaller class

↳ Def. Let $(X, \|\cdot\|)$ be a normed space and $A \subset X$. We say that A is dense in X if

$$\forall f \in X \quad \forall \epsilon > 0 \quad \exists g \in A \text{ s.t. } \|f - g\| < \epsilon.$$

↳ In our case: $X = C([0,1])$, $A = \{\text{piecewise lin. func. on } [0,1]\}$; $\|\cdot\| = \|\cdot\|_\infty$

- Therefore, shallow scalar ReLU networks can approximate any cont. func. on $[0,1]$ arbitrarily well.

Important:

↳ Universality is a theoretical existence statement. It says that good approximation exist, not that they are easy to find or efficient to represent.

↳ The key idea behind universality is not specific to ReLU. What really matters is that the activation function is

- nonlinear
- not a polynomial
- sufficiently expressive to generate rich function classes under lin. comb. and compositions.

How far does our result generalize?

The $f: [0,1]^n \rightarrow \mathbb{R}$, $n \geq 2$ case

What changes

↳ piecewise lin. functions are defined on polyhedral regions (polytopes)

↳ a piecewise lin. function is affine on each polytope / $f(x) = A_i x + b_i$ /

↳ ReLU networks still produce ~~piece~~ piecewise lin. func.

What is different

↳ Shallow networks generate only limited types of partitions. Depth is needed to create sufficiently rich polyhedral decompositions. — 6 —

So, all in all, the approximation idea survives but universality in higher dimensions requires depth. This is why we need deep networks.

The $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ case

Any vector valued function can be rewritten componentwise: $f(x) = (f_1(x), \dots, f_m(x))$.

↳ If each $f_j(x)$ can be approximated uniformly, then f can be approx. uniformly

↳ MNs implement this naturally via multiple output neurons.

Therefore universality results extend componentwise here and deep networks approximate f by composing simple piecewise linear maps.

So our 1D universality result extends to higher dimensions and vector valued functions but depth becomes essential to achieve sufficient expressive power.

What does universal approximation really mean?

Universality ~ it is a representation statement

A given class of MNs is dense in a target space

Approximation ~ it concerns how the representation is achieved

In general, optimization theory tells us whether such representations can actually be found. Basically universal approx. thm says "a good network exists" and optimization theory asks "can we find it by training"? Opt. theory here typically studies

↳ the loss landscape

↳ existence of local vs global minima

↳ convergence of training algorithms

↳ whether reasonable initialization and training dynamics lead to a good solution.

So a network may be expressive enough but training may get stuck or require unrealistic time or data.

Section 3: Optimization and training

Optimization via gradient descent

Consider the total loss over the dataset for a given choice of parameters

$$L_{\text{total}}(w) = \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} \ell(F(x; w), y)$$

Remarks:

• $\mathcal{D} = \{(x_i, y_i) \in X \times Y\}_{i=1}^N$

• Multiplying the loss by $1/N$ doesn't change the minimizer, only the scale of loss and its gradient.

• For the sake of simplicity $w \in \mathbb{R}^p$.

Reminder from Multivariate Calculus

Assume that $\nabla f_P \neq 0$. Let u be a unit vector making an angle Θ with ∇f_P . Then $D_u f(P) = \|\nabla f_P\| \cos(\Theta)$.

• ∇f_P points in the direction of max. rate of increase of f at P

• $-\nabla f_P$ ——— || ——— max. rate of decrease ——— || ———

• ∇f_P is normal to the level curve (or surface) of f at P .

Thus, in our setting the gradient $-\nabla L_{\text{total}}(w) = -\left(\frac{\partial L_{\text{total}}}{\partial w_1}, \dots, \frac{\partial L_{\text{total}}}{\partial w_p} \right)$

Assuming differentiability, this motivates the gradient descent iteration (GD iteration)

$$w^{(k+1)} = w^{(k)} - \eta \nabla \mathcal{L}_{\text{total}}(w^{(k)}), \text{ where } \eta > 0.$$

Remarks:

→ η is the step size $\sim$ learning rate

→ an update from k to $k+1 \sim$ 1 epoch, i.e. 1 iteration using the entire dataset

→ GD is a local method and the loss landscape may have

- multiple local minima
- saddle points
- no global minimum at all

Thus, convergence to a global minimum is not guaranteed.

When the parameter dimension p is large, or the dataset size N is large, computing the full gradient becomes computationally expensive.

Idea: approximate the gradient using a random subset, a so-called mini-batch. Denote a batch by an index set $I \subset \{1, \dots, N\}$, then we consider the loss over that single batch

$$\mathcal{L}_I(w) = \frac{1}{|I|} \sum_{i \in I} \mathcal{L}(f(x_i, w), y_i).$$

Let's say we divide the dataset into batches $I_1, I_2, \dots, I_B$. We can split then our GD step into B smaller steps:

$$w^{(k+1)} = w^{(k)} - \eta \nabla \mathcal{L}_{I_k}(w^{(k)})$$

mini-batch GD

where the initial parametrization $w^{(0)}$ is given.

Why is this relevant?

Because if the batch I is uniformly drawn, then $E(\nabla \mathcal{L}_I(w)) = \nabla \mathcal{L}_{\text{total}}(w)$, i.e. the gradient of a uniformly drawn batch is an unbiased estimator of the gradient of the full dataset.

Lemma: $E(\nabla \mathcal{L}_I(w)) = \nabla \mathcal{L}_{\text{total}}(w)$

Proof: Let $|I| = m$ be fixed size. Since I is uniformly drawn at random, so it means that each index $i \in \{1, \dots, N\}$ has the probability (same) of being included in I : $P(i \in I) = \frac{m}{N}$

$$\text{So } E(\nabla \mathcal{L}_I(w)) = E\left(\frac{1}{m} \sum_{i \in I} \underbrace{\nabla \mathcal{L}(f(x_i, w), y_i)}_{\substack{\text{short notation} \\ \mathcal{L}_i(w)}}\right) = \frac{1}{m} \sum_{i=1}^N E\left(\underbrace{\mathbb{1}_{\{i \in I\}}}_{\substack{\text{lin. of} \\ \text{expectation}}} \nabla \mathcal{L}_i(w)\right) =$$

$$* \mathbb{1}_{\{i \in I\}} = \begin{cases} 1 & \text{if } i \in I \\ 0 & \text{otherwise} \end{cases} \quad \text{This is the same as the original sum over } i \in I \text{ because all elements not in the batch contribute 0.}$$

$$= \frac{1}{m} \sum_{i=1}^N E(\mathbb{1}_{\{i \in I\}} \nabla \mathcal{L}_i(w)) = \frac{1}{m} \sum_{i=1}^N \frac{m}{N} \nabla \mathcal{L}_i(w) = \frac{1}{N} \sum_{i=1}^N \nabla \mathcal{L}_i(w) = \frac{1}{N} \sum_{i=1}^N \nabla \mathcal{L}(f(x_i, w), y_i) = \nabla \mathcal{L}_{\text{total}}(w).$$

we use that if random var X is an indicator, then $E(cX) = cE(X)$, here $c = \nabla \mathcal{L}_i(w)$, $X = \mathbb{1}_{\{i \in I\}}$ and $\square$

$E(X) = \sum_{x \in \text{range}(X)} x \cdot P(X=x) = 0P(X=0) + 1P(X=1) = P(X=1)$, so for this discr. rand. var. we have

$$E(\mathbb{1}_{\{i \in I\}}) = P(\mathbb{1}_{\{i \in I\}}=1) = P(i \in I) \text{ since } X = \mathbb{1}_{\{i \in I\}}, X=1 \Leftrightarrow i \in I; X=0 \Leftrightarrow i \notin I$$

$$\text{In our case } P(i \in I) = \frac{m}{N}$$

Let's have a practical summary.

- ↳ GD
- batch size = full dataset
 - 1 epoch = 1 iteration
 - less noisy but expensive

- ↳ Mini-batch GD:
- 1 iteration = 1 mini-batch processed + weight and bias (w) update
 - 1 epoch = all batches processed once
 - Most modern DL implementation use this

Remarks:

↳ Stochastic GD (SGD)

- Batch size = 1
- Very noisy but helps escape local minima

↳ Mini-batches are disjoint: each sample appears exactly once per epoch

↳ Typically we shuffle the dataset before each epoch: batches are random

↳ Typically there are multiple iterations per mini-batch

Training

In practice this involves more than applying mini-batch GD on the whole dataset. The dataset is the only information available about the unknown function we aim to approximate, so the quality of our model can be evaluated through this data. Since NNs are universal approximators, they can always be made to fit a given dataset perfectly which makes over-fitting a real risk. Our true objective, however, is generalization: enabling the network to produce accurate outputs for inputs it has not encountered during training.

Train - test split

We address this by splitting the dataset: one part is used for training while the unseen portion (test) is used to evaluate generalization. The exact ratio depends on the task but 80% - 20% split is a common rule of thumb.

$$\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}} \subset \mathcal{D}$$

Perform mini-batch GD on the training dataset and judge the performance of our network by testing loss:

$$L_{\text{train}}(w) := \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} \ell(f(x,w), y) \leadsto L_{\text{test}}(w) := \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x,y) \in \mathcal{D}_{\text{test}}} \ell(f(x,w), y)$$

Monitoring the test loss during training helps assess generalization, determine training duration and detect overfitting / see the plot!

Train - validation - test split

When tuning hyperparameters

↳ user specified setting that are not learned during training
~ model size, activation function, data split ratio, batch size, etc.

Evaluating performance repeatedly on the test set leads to overfitting to the dataset. To avoid this the data should be split into 3 parts:

- Training set - used to fit model parameters
- Validation set - used for hyperparameter tuning and model selection
- Test set - used only once for the final evaluation

The test set should remain untouched during model development in order to provide an unbiased estimate of generalization performance. If the dataset is small, cross-validation can be used for model selection while still keeping a separate final test set.

II Practical aspects

In this part we turn to the practical aspects of training NNs while viewing the arising phenomena and methods from a mathematical perspective.

Section 1: Further activations and their roles

Some activation functions directly yield universal approximation results. Other commonly used operations do not increase expressive power but preserve density once universality is achieved. These can still be useful in practice providing improved generalization, preserving robustness or making computations easier.

We formalize this density property.

Theorem: Let $(X, \|\cdot\|)$ be a normed space and let $A \subset B \subset X$. If A is dense in X , then B is dense in X .

Proof: Let $f \in X$ and $\epsilon > 0$ be arbitrary. To show B is dense in X we need $b \in B: \|f - b\| < \epsilon$.

Since A is dense in $X \exists a \in A: \|f - a\| < \epsilon$. Since $A \subset B$, the element $a \in A$ also belongs to B .

Let $b := a \in B$. Then $\|f - b\| = \|f - a\| < \epsilon$. Since f and ϵ were arbitrary and we found $b \in B$:

$\|f - b\| < \epsilon$, therefore $\forall f \in X \forall \epsilon > 0 \exists b \in B: \|f - b\| < \epsilon$. $\square$

Let's examine exactly what the choices for A and B could be in our context.

A = shallow universal activation based network

↳ Heaviside

In practice: no meaningful gradient exists so GD cannot optimize

↳ ReLU

In practice: nondiff. at $x=0$ but most of the implementation defines this 0 or 1

pos. region: works well w/ grad. propagation. There is no saturation and so there is no vanishing gradient problem.

Remark: Saturation in activations (outputs flattening for large inputs) causes very small derivatives which multiply across layers during backpropagation, consequence: early layers learn extremely slow or not at all.

neg. region: dying ReLU

neuron input $< 0 \rightarrow$ output $= 0 \rightarrow$ grad $= 0 \rightarrow$ weights stop updating $\rightarrow$ neuron inactive $\rightarrow$ If many neurons die the NN's capacity decreases so we have slower or weaker learning.

Overcome this dying issue:

• Leaky / Parametric ReLU: $f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}, \alpha > 0$

Allows a small gradient for negative inputs $\rightarrow$ neuron can still learn

• Careful weight initialization

Chooses weights so that most neurons start with pos. inputs $\rightarrow$ reduced change of neurons being inactive at the beginning

↳ Sigmoid and hyperbolic tangent

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

/ see the plot /

/ see the plot /

History: sigmoid comes from logistic regression $\rightarrow$ it is smooth and $0 \leq \sigma(x) \leq 1$ so interpretable as a probability. Tanh is the scaled version (range -1 to 1).

In practice: less preferred nowadays since they saturate easily so we have vanishing gradients. Still can be okay for small or shallow networks but DL mostly avoids them.

Options to overcome the issue

- weight initialization which gives larger gradients
- normalization layers: keeps activations centered and scaled which reduces saturation

$\rightarrow$ Swish / SiLU

This is the smooth variant of ReLU

$$\text{Swish}(x) = x \sigma(\beta x) = \frac{x}{1 + e^{-\beta x}} \quad \beta > 0 \quad \text{If } \beta = 1 \text{ it is the sigmoid weighted linear unit,}$$

/ see the plot /

i.e. SiLU.

In practice: It is smooth and non saturating and works well for deep NNs.

Minor issues: • slightly more computationally expensive than ReLU (requires sigmoid computation)

• still needs good initialization for very deep NNs.

• $f'(x) = 0$ only for some neg. values of x . Grad almost always > 0 . Overcome issue: weight initialization $\rightarrow$ input stays in regions with grad

Swish is one of the state of the art activations in modern DL often preferred over ReLU for very deep or residual networks but ReLU remains popular for simplicity and speed.

B = networks from A + neutral operations

$\rightarrow$ Dropout

Randomly deactivates some neurons during training.

$$(\text{dropout}(x))_i = \begin{cases} 0 & \text{with prob. } p \\ x_i & \text{with prob. } 1-p \end{cases}$$

In practice: helps regularization and prevents overfitting

Potential drawback: introduces noise into outputs

$\rightarrow$ Max-pooling

Applies a local maximum over a region of activations

$$(\text{maxpool}(x))_{i,k} = \max(x_i, x_{i+1}, \dots, x_{i+k-1})$$

In practice: reduces dimensionality (fewer parameters $\rightarrow$ faster computation and a regularization effect) and helps local invariance (robust to small local shifts or noise $\rightarrow$ preserve essential patterns)

Potential drawback: may remove fine grained information

$\rightarrow$ Normalization (Batch Norm or Layer Norm)

Batch Norm: layer with N neurons and a mini batch of B samples

$$\forall i = 1, \dots, N \rightsquigarrow \underbrace{\mu_i = \frac{1}{B} \sum_{j=1}^B x_i^{(j)}}_{\text{mean}} \quad \text{and} \quad \underbrace{\sigma_i^2 = \frac{1}{B} \sum_{j=1}^B (x_i^{(j)} - \mu_i)^2}_{\text{variance}}$$

$$\text{Then } \hat{x}_i^{(j)} = \frac{x_i^{(j)} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \rightsquigarrow y_i^{(j)} = \underbrace{\hat{x}_i^{(j)}}_{\substack{\text{output} \\ \text{mean } 0, \text{ var } 1}} \underbrace{\alpha_i x_i^{(j)} + \beta_i}_{\substack{\alpha_i, \beta_i \in \mathbb{R} \text{ learnable parameters} \\ \uparrow \quad \uparrow \\ \text{scale} \quad \text{shift} \rightarrow \text{allow the network to adjust}}}$$

for preventing division by 0

Intuitively: making numerical stability

ϵ is really small $\approx 10^{-8}$

the normalized outputs for optimal learning across the batch

"across the batch per neuron"

In practice: allows higher learning rates, stabilizes training, implicit regularization via batch stochasticity

Drawbacks: depends on batch size $\rightarrow$ small batches may reduce effectiveness

introduces dependency between samples in a batch

Layer Norm: single sample passing through a layer of N neurons

$$\mu_L = \frac{1}{N} \sum_{i=1}^N x_i, \quad \sigma_L^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_L)^2 \rightarrow \hat{x}_i = \frac{x_i - \mu_L}{\sqrt{\sigma_L^2 + \epsilon}}, \quad y_i = \gamma_i \hat{x}_i + \beta_i$$

γ, β across neurons per sample

In practice: batch-independent, stabilizes neuron distribution for each sample

Drawbacks: slightly higher computational cost

lacks stochastic regularization from batch averaging

doesn't exploit batch statistics so convergence can be slower $\rightarrow$ due to normalization per sample

BatchNorm introduces slight norm which provides implicit regularization. LayerNorm lacks this effect.

neuron outputs are less stable which can make learning over slightly slower

Important: the discussed density A and B reasoning can be applied layer by layer in deep networks.

Output activations

Some nonlinearities are used only in the output layer to impose structural constraints. Let's list 3 common choices:

activation	output range	typical use
sigmoid	$(0,1)$	binary prob.
tanh	$(-1,1)$	binary regression
softmax	prob. vector $x_i > 0, \sum x_i = 1$	multiclass regression classification

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1}^C e^{x_j}} \quad i=1, \dots, C$$

Sigmoid and tanh are universal approximators on $C([a,b])$. Lin. comb. prevent the range restriction what we can see as an output layer (For sigmoid it is $(0,1)$).

Softmax, however, is not a universal approximator in alone. On the other it fits to the discussed density A and B reasoning. A is a proper universal approximation compatible structure even in deep NN case and B is the softmax transformation output so we preserve the density property

In practice: ideal for multiclass classification, fully differentiable $\rightarrow$ works well with backpropagation

Drawbacks: large logits (logit $\sim$ raw scores produced by the last affine layer before the output layer) can cause overflow if not normalized

not ideal for multi-label tasks (classes are not independent) $\rightarrow$ use sigmoid per class instead of softmax

Section 2: Scaling to high dimensional data

Let us consider a feedforward NN:

$\mathcal{N}: \mathbb{R}^{N_0} \rightarrow \mathbb{R}^{N_{L+1}}$ given by $\mathcal{N}: F_{L+1} \circ G_L \circ F_L \dots \circ G_1 \circ F_1$ where $F_i(x) = A_i x + b_i$ where $A_i \in \mathbb{R}^{N_i \times N_{i-1}}$ and $b_i \in \mathbb{R}^{N_i}$, $i=1, \dots, L+1$ and $G_i, i=1, \dots, L$ are appropriate activation functions.

Layer i is called fully connected (dense) if every entry of A_i is a trainable parameter, i.e. each output component depends on all input components. / The bias b_i doesn't create connections between inputs and outputs it only shifts each output component independently. /

The NN is fully connected if this holds for all layers. In practice, the general form is restricted: architectures arise by constraining the parameter space so that only a subset of the entries of A_i and b_i are trainable, while the remaining ones are fixed.

The primary reason fully connected NNs are not used is that they simply don't scale to high dimensional data.

For example: for a 1000×1000 RGB image

$x \in \mathbb{R}^{3 \times 1000 \times 1000} \Rightarrow d = 3 \cdot 10^6$. A fully connected linear map requires

$A \in \mathbb{R}^{d \times d}$, $d^2 = 9 \cdot 10^{12}$ parameters which would require approximately 72 terabytes in 64 bit precision which makes fully connected architectures impractical for high dimensional data such as images.

To reduce the massive parameter count in high dimensional fully connected layers, three strategies are used:

↳ Sparsity: most matrix entries are fixed to zero, reducing storage and computation

↳ Weight sharing: the same parameter is reused across multiple locations, so only unique values are stored. This is a special case of parametrization.

↳ Parametrization: matrix entries are computed from a smaller set of parameters storing only the parameters and generating the full matrix as needed.

Example:

$\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}$
fully connected

$\begin{pmatrix} a & 0 & 0 \\ 0 & 0 & 0 \\ 0 & b & 0 \end{pmatrix}$
sparse

$\begin{pmatrix} a & b & b \\ b & a & b \\ b & b & a \end{pmatrix}$
shared weights

$\begin{pmatrix} f_1(a,b) & \dots & f_3(a,b) \\ \vdots & & \vdots \\ f_4(a,b) & \dots & f_9(a,b) \end{pmatrix}$
parametrized

fully connected: stores 9 numbers

other: only 2 numbers

$f_i(a,b) = \sin(a) + \exp(b)$
 $\forall i$, it can be different functions for i

These strategies are not mutually exclusive, so a single layer or architecture can use more than 1 strategy at the same time.

Example:

$\begin{pmatrix} f_1(a,b) & 0 & a \\ 0 & a & 0 \\ a & 0 & f_2(a,b) \end{pmatrix}$
□ sparse
□ shared
□ parametrized

Section 3: Vanishing and exploding gradients

Let us consider the composition of an activation function n times, i.e.

$$F_n(x) = \underbrace{G \circ G \circ G \dots \circ G}_n(x)$$

Then, $F'_n(x) = \prod_{j=0}^{n-1} G'(G^{(j)}(x))$. Let's investigate this in detail for two well-known activation functions.

Vanishing gradients

Let's start with the sigmoid case. Then $G(x) = \frac{1}{1+e^{-x}}$ so $G'(x) = G(x)[1-G(x)]$. Then we have the bounds for $G'(x)$ as $0 < G'(x) \leq \frac{1}{4}$. /see the plot/

Therefore $|F'_n(x)| \leq (\frac{1}{4})^n$ so $F'_n(x)$ tends to 0 exponentially fast.

Why sigmoid is structurally problematic

Because it saturates $G(x) \rightarrow 0$ or 1 ($x \rightarrow \pm\infty$) and both cases $G'(x) \rightarrow 0$ ($x \rightarrow \pm\infty$)

So the gradients become smaller and smaller in a deep network ~ vanishing gradient problem.

Remark: In theory, we could train longer to compensate for very small gradients. However, in practice this fails due to floating point arithmetic.

For floating point numbers $a + b = a$ if $|b| \ll |a|$, so sufficiently small parameter updates have no effect at all.

Let's investigate the $G(x) = \text{ReLU}(x) = \max(0, x)$ case. Then $G'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$

If all intermediate activations stay positive: $F'_n(x) = 1$ so there is no decay in the active region ($x > 0$). ReLU can fail differently. If activation becomes negative $\leadsto G'(x) = 0$ so the gradient becomes exactly zero.

This is dead ReLU not exponential vanishing. /see the plot/

Suggestion: Leaky / Parametric ReLU $(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0, \alpha > 0 \end{cases}$

Natural question: does small α reintroduce vanishing gradients?

Let's recall that $F'_n(x) = \prod_{j=0}^{n-1} G'(G^{(j)}(x))$. If all activations are negative, then $F'_n(x) = \alpha^n$. So if

$\alpha = 0.01$, then $\alpha^n = 10^{-2n}$ which is an exponential decay. Thus, in a purely negative chain it behaves like the sigmoid vanishing. Why this is still better than sigmoid because this vanishing is conditional not structural.

Why does it still work well in practice?

- $\hookrightarrow$ activations are not all negative
- $\hookrightarrow$ special initialization keeps variance stable
- $\hookrightarrow$ BatchNorm re-centers activations
- $\hookrightarrow$ NNs are not simple repeated compositions of the same functions

} So the catastrophic α^n effect rarely happens.

Remark: A small α can be problematic if the NN is deep, or we don't have normalization or many neurons stuck negative

In practice we don't choose large α ($\alpha \approx 1$) because

- $\hookrightarrow$ activation becomes almost linear / slopes for $x < 0$ and $x > 0$ are nearly the same ~ behaves like a lin. func. /
- $\hookrightarrow$ expressivity decreases
- $\hookrightarrow$ network loses sparsity structure (it happens for $\alpha > 0$)

α must balance:

- $\hookrightarrow$ avoid dead neurons
- $\hookrightarrow$ preserve nonlin.
- $\hookrightarrow$ maintain gradient flow

} Empirically $\alpha = 0.01$ works well

Remark: General take home message: for a deep NN: Total gradient $\| \prod_{j=0}^{n-1} J_j \| \leq \prod_{j=0}^{n-1} \| J_j \|$

So if $\| J_j \| < 1 \rightarrow$ vanishing gradient
if $\| J_j \| > 1 \rightarrow$ exploding gradient

Exploding gradients

Based on the previous general take home message ReLU causes exploding gradients but not because of the activation itself. It happens because of the weight matrices.

Let's have

$$x^{(j+1)} = \text{ReLU}(A_j x^{(j)} + b_j)$$

Then the Jacobian in the active region is $J_j = D_j A_j$ where D_j is a diagonal matrix with entries 0 or 1. Therefore $\|J_j\| \approx \|A_j\|$, so the total gradient behaves like: $\|J\| \approx \frac{1}{\delta} \|J_j\|$

If the norms are larger than 1 gradients explode.

This problem strongly depends on the architecture.

↳ Shallow NNs: rare if initialization is correct

↳ Very deep fully connected NN: possible especially without proper initialization

On the other hand it is less common with modern ReLU NNs since several stabilizing mechanisms (initializations or Batch Normalization) are now standard.

In general modern practices try to ensure that $\|A_k\| \approx 1$ so the product will be remained controlled.